

Birla Institute of Technology and Science Pilani

Hyderabad Campus

CDRF Project



Mobile Application: Harnessing Technology to empower Informal female entrepreneurs-Telangana Insights

Team:

1. Swati Alok
2. Subhrakanta Panda
3. Mou Jana
4. Pratyush Nair
5. Navya Garg

Tentative Outline of the Steps:

To develop a mobile application with the proposed features, tentative steps to be followed:

1. Define Requirements & Plan UI/UX

- **User Research:** Identify user needs and pain points to ensure the app remains intuitive and useful. (Tailoring, Beauty Parlour, Tiffin Centre, Vegetables and fruit sellers, Stationary shop)
- **Wireframing & Prototyping:** Design a simple, tab-based navigation system with a clean UI. Use tools like Figma or Adobe XD.
- **Accessibility & Localization:** Ensure the app supports multiple languages and easy readability.

2. Choose the Tech Stack

- **Frontend:** Flutter (Dart) or React Native (JavaScript) for cross-platform development.
- **Backend:** Node.js with Express or Firebase for real-time database updates.
- **Database:** Firestore (for scalability) or SQLite (for offline support).
- **Authentication:** Firebase Authentication or OAuth for secure login.

3. Implement Core Features

a) Inflow/Outflow Management

- **User Input:** Allow manual entry of income/expenses.
- **Visualization:** Implement graphs and reports using chart libraries like Chart.js or Recharts.
- **Summarization:** Provide daily, weekly, and monthly reports.

b) Stock Assessment

- **Inventory Management:** CRUD operations for stock items.
- **Notifications:** Set alerts for low stock levels.
- **Reports:** Generate stock usage reports to track trends.

c) Customized Scheduler

- **Calendar Integration:** Allow daily, weekly, and monthly views.
- **Reminders & Notifications:** Send push notifications for upcoming appointments.
- **Customization:** Tailor scheduling based on different business needs.

d) Customer Management

- **Customer Profiles:** Store customer details, purchase history, and preferences.
- **Promotions & Discounts:** Notify customers about special offers.
- **Feedback Collection:** Simple review or rating system for customer experience tracking.

4. Testing & Deployment

- **Usability Testing:** Gather feedback from real users to improve the app's experience.
- **Performance Optimization:** Ensure fast load times and minimal lag.
- **Deployment:** Publish on Google Play Store.

5. Future Enhancements

- AI-based recommendations for stock management.
- Integration with digital payment solutions.
- Multi-user support for small teams.

Scope for Improved Outline for Mobile Application Development

1. Define Requirements & Plan UI/UX

Enhancements: Break this step into clearer subcategories for better organization and completeness.

- **User Research & Competitor Analysis**
 - Identify user needs and pain points through surveys and interviews.
 - Analyze similar apps to identify gaps and opportunities.
 - **Wireframing & Prototyping**
 - Design a simple, tab-based navigation system with a clean UI.
 - Use tools like **Figma, Adobe XD, or any other** for wireframing.
 - Create interactive prototypes to test user flows before development.
 - **Accessibility & Localization**
 - Ensure support for **multiple languages and currency formats**.
 - Follow **WCAG (Web Content Accessibility Guidelines)** for usability.
 - **Feature Prioritization (MVP Planning)**
 - Define **must-have vs. nice-to-have features** to release the app faster.
-

2. Choose the Tech Stack

Enhancements: Include additional recommendations for scalability and efficiency.

- **Frontend:**
 - **Flutter (Dart)** – Better performance & UI consistency.
 - **React Native (JavaScript/TypeScript)** – Good for reusing web code.
 - **Backend:**
 - **Node.js with Express** – Scalable backend with RESTful APIs.
 - **Firebase (Serverless)** – Real-time database & easy authentication.
 - **Database:**
 - **Firestore (NoSQL, real-time sync)** – Best for scalability.
 - **SQLite (Local storage)** – Useful for offline mode.
 - **PostgreSQL or MySQL** – If structured data is needed.
 - **Authentication:**
 - **Firebase Authentication** – Easy integration.
 - **OAuth (Google, Facebook, Apple Login)** – Better user experience.
 - **Other Integrations:**
 - **Push Notifications:** Firebase Cloud Messaging (FCM).
 - **Payment Gateway:** Stripe, Razorpay, or PayPal (Future upgrade) or anything else.
-

3. Implement Core Features

Enhancements: Add **error handling, offline mode, and data export** features.

a) Inflow/Outflow Management (Cash Flow Tracking)

- **User Input:**
 - Manual entry for income and expenses.
 - **Offline mode:** Data syncs when back online.
- **Visualization & Summarization:**
 - Implement graphs and reports using **Chart.js or Recharts**.
 - Provide daily, weekly, and monthly reports.
- **Data Export & Backup:**
 - Allow users to export financial reports in **PDF or Excel**.

- Cloud backup option for data security.

b) Stock Assessment (Inventory Management)

- **Inventory Tracking:** CRUD operations for stock items.
- **Notifications & Alerts:** Low stock alerts via push notifications.
- **Auto Reordering Suggestion:** AI-based recommendations for restocking.

c) Customized Scheduler (Appointments & Reminders)

- **Calendar Integration:** Show daily, weekly, and monthly views.
- **Reminders & Notifications:** Push notifications and SMS reminders.
- **Customization:** Tailor scheduling for different business types.

d) Customer Management

- **Customer Profiles:** Store customer details, preferences, and purchase history.
- **Loyalty & Promotions:** Notify customers about special offers & discounts.
- **Feedback System:** Collect customer ratings and reviews.

4. Testing & Deployment

Enhancements: Include **security testing, beta testing, and continuous integration (CI/CD)**.

- **Functional & Usability Testing:**
 - Conduct tests using **Firebase Test Lab (Android)** or any other available platform/tool.
 - Perform A/B testing on different UI designs.
- **Security & Performance Optimization:**
 - Encrypt sensitive user data.
 - Optimize app load times with efficient caching and database indexing.
- **Beta Testing & User Feedback:**
 - Use **Google Play Beta Program & TestFlight** for iOS.
 - Gather feedback from real users before public launch.
- **Deployment & CI/CD Setup:**
 - Automate deployments with **GitHub Actions** or **Bitrise**.
 - Publish the app on **Google Play Store**.

5. Future Enhancements

Enhancements: Organize future upgrades into **short-term and long-term** improvements.

Short-Term Enhancements (Next Releases)

- **AI-based stock & expense predictions** using machine learning.
- **Integration with digital payments (Razorpay, Stripe, PayPal).**
- **Multi-user support (for teams or partners).**

Long-Term Enhancements

- **Voice command support** for hands-free usage.
- **WhatsApp/Email invoice sending feature** for customers.
- **Multi-platform support (Web & Desktop version).**

Detailed Breakdown:

1. App Architecture & Tech Stack

Frontend (Mobile App)

- **Framework:** Flutter (Dart) / React Native
- **State Management:** Provider (Flutter) / Redux or Context API (React Native)
- **UI Components:** Material Design (Android) & Cupertino (iOS)

Backend

- **Database:** Firebase Firestore (real-time sync) or PostgreSQL (structured data)
- **Cloud Functions:** Firebase Functions / Node.js with Express for backend logic
- **Authentication:** Firebase Auth (email/password, phone, social login)

Third-Party Integrations

- **Push Notifications:** Firebase Cloud Messaging
 - **Analytics & Crash Reporting:** Google Analytics for Firebase
 - **Payment Integration (Future Upgrade):** Razorpay / Stripe
-

2. Detailed Feature Implementation

A. Inflow/Outflow Management (Cash Flow)

Objective: Simplify income & expense tracking for easy financial decision-making.

Implementation:

1. **Manual Entry:**
 - Add income (sales, services) and expenses (supplies, rent, utilities).
 - Categorization (e.g., Business Income, Expenses, Miscellaneous).
2. **Data Visualization:**
 - Monthly charts (bar, pie) to display trends.
 - A summary page showing profit/loss.
3. **Automation (Future Upgrade):**
 - Scan receipts using OCR (Google Vision API).
 - Auto-categorization of transactions.

Sample UI Flow:

Tab: "Cash Flow" → Click "+" to add income/expense → Input details → Save → View summary in graphs.

B. Stock Assessment (Inventory Management)

Objective: Avoid stockouts and optimize inventory levels.

Implementation:

1. **Stock Addition & Tracking:**
 - Add new stock items (product name, quantity, cost, supplier).
 - Set minimum stock levels.
2. **Alerts & Reports:**
 - Notify users when stock is running low.
 - Generate stock summary reports (weekly/monthly).
3. **Sales & Expense Linkage:**
 - Deduct sold stock from inventory when a sale is recorded.
 - Calculate profit margins per item.

Sample UI Flow:

Tab: "Inventory" → View stock list → Click "Add Stock" → Input details → Save → Set reorder level notifications.

C. Customized Scheduler

Objective: Help users efficiently plan customer appointments.

Implementation:

1. **Appointment Scheduling:**
 - Add/edit/cancel appointments.
 - Select day-wise view (for beauty salons) or range-based (for tailors).
2. **Reminders & Notifications:**
 - Send reminders via push notifications.
 - Daily schedule summary in the morning.
3. **Customer Integration:**
 - Link appointments to customer profiles.
 - View appointment history for repeat customers.

Sample UI Flow:

Tab: "Scheduler" → View calendar → Click "+" to add an appointment → Select customer & time → Set reminder → Save.

D. Customer Management

Objective: Track customer interactions, preferences, and purchases.

Implementation:

1. **Customer Profiles:**
 - Save customer details (name, contact, preferences).
 - Track past purchases & service history.
2. **Notifications & Discounts:**

- Send promotions via in-app messages.
 - Offer loyalty discounts.
3. **Feedback System:**
- Customers can provide ratings & comments.
 - Generate reports on customer engagement.

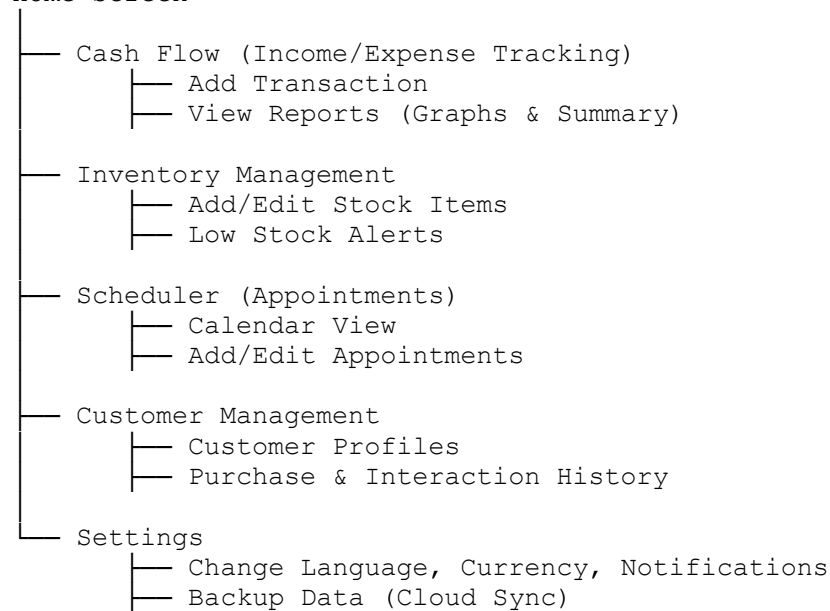
Sample UI Flow:

Tab: "Customers" → View list → Click on a customer → See purchase history → Send promotions or discounts.

3. UI Flow Diagram

Here's an overview of how the user will navigate through the app:

Home Screen



4. Deployment Plan

Phase 1: MVP (Minimum Viable Product)

- Basic UI with cash flow, inventory, scheduler & customer management.
- Manual transaction entry.
- Local storage (SQLite) or Firebase for database.

Phase 2: Enhanced Features

- Push notifications & stock alerts.
- Report generation (PDF/Excel).
- Basic analytics dashboard.

Phase 3: Advanced Automation

- AI-based sales predictions.
- Receipt scanning with OCR.
- Payment gateway integration.

Sample Wireframe

Here's a basic wireframe layout for each major section of the app:

Home Screen (Dashboard)

- **Top Section:** User greeting, quick summary of cash flow (income/expense balance).
- **Main Sections:**
 - **Cash Flow** → Track earnings & expenses
 - **Inventory** → Manage stock levels
 - **Scheduler** → View & set appointments
 - **Customers** → Manage customer profiles

Cash Flow Screen

- **Transaction List:** Displays recent transactions.
- **Add Button (+):** Opens form to input income or expense.
- **Reports Tab:** Graphs & summaries.

Inventory Screen

- **Stock List:** Shows available stock & low-stock alerts.
- **Add/Edit Button:** Allows users to modify stock levels.
- **Filter Option:** View by categories (e.g., sold, available).

Scheduler Screen

- **Calendar View:** Daily/weekly/monthly view of appointments.
- **Add Button (+):** Schedule a new appointment.
- **Reminder Toggle:** Enable/disable notifications.

Customer Management Screen

- **Customer List:** View customer details & history.
- **Search Bar:** Find customers by name or phone number.
- **Profile Page:** Track purchases & offer discounts.

Mobile App Development Roadmap

Estimated Timeline: 5-6 Months

Phase 1: Research & Planning (Weeks 1-3)

Goals:

- Define requirements, user personas, and core features.
- Create wireframes and prototypes.
- Finalize the tech stack and database structure.

Tasks Checklist:

- ✓ Conduct **user research** (interviews, surveys).
- ✓ Define **user personas & pain points**.
- ✓ Sketch initial wireframes using **Figma or Adobe XD**.
- ✓ Create a **low-fidelity prototype** for early feedback.
- ✓ Select tech stack (**Flutter/React Native, Firebase/Node.js**).
- ✓ Plan **database schema & API endpoints**.
- ✓ Draft **project timeline & team roles**.

Deliverables:

- Wireframes & user flows.
 - Tech stack decision document.
 - Database schema design.
 - Development timeline.
-

Phase 2: UI/UX Design & Prototyping (Weeks 4-6)

Goals:

- Build a high-fidelity prototype.
- Define UI components and branding.
- Gather feedback from test users.

Tasks Checklist:

- ✓ Create a **high-fidelity prototype** with interactive UI.
- ✓ Design UI components (**buttons, cards, icons, forms, color scheme, typography**).
- ✓ Ensure **accessibility and localization** (multi-language & currency support).
- ✓ Conduct **user testing** on the prototype.
- ✓ Make UI/UX refinements based on feedback.

Deliverables:

- Final UI design in **Figma/Adobe XD**.
 - UI component library & style guide.
 - Interactive prototype for validation.
-

Phase 3: Backend & API Development (Weeks 7-10)

Goals:

- Build backend services and API endpoints.
- Set up database & authentication.
- Implement basic security measures.

Tasks:

- ✓ Set up **Node.js + Express** (or Firebase) backend.
- ✓ Configure **Firestore/PostgreSQL/SQLite** database.
- ✓ Implement **user authentication (Firebase Auth/OAuth)**.
- ✓ Develop **CRUD APIs** for transactions, inventory, appointments, customers.
- ✓ Set up **cloud functions for push notifications**.
- ✓ Implement **data encryption & security protocols**.

Deliverables:

- Fully functional API endpoints.
 - Database schema & cloud storage setup.
 - Authentication & authorization system.
 - Security measures (token-based authentication, encryption).
-

Phase 4: Frontend Development (Weeks 11-14)

Goals:

- Develop core UI features.
- Connect frontend with backend APIs.
- Ensure smooth navigation and data management.

Tasks:

- ✓ Set up **Flutter/React Native project structure**.
- ✓ Implement **tab-based navigation** for different modules.
- ✓ Build UI components for:
 - **Cash Flow Management** (income/expense tracking).
 - **Inventory System** (add, update, alert notifications).
 - **Scheduler** (appointments, calendar view, reminders).
 - **Customer Management** (profiles, purchase history).Integrate **backend APIs** for data fetching and storage.
Add **basic error handling & validation**.

Deliverables:

- Fully functional frontend integrated with backend.
 - Responsive design & smooth UI interactions.
 - Working navigation system.
-

Phase 5: Feature Enhancements & Integrations (Weeks 15-17)

Goals:

- Implement notifications, reports, and payment integration.
- Optimize app performance.
- Add data backup & export functionality.

Tasks:

- ✓ Implement **push notifications (Firebase Cloud Messaging)**.
- ✓ Integrate **report generation (Chart.js, Recharts)**.
- ✓ Add **data export options (CSV, PDF reports)**.
- ✓ Implement **payment integration (Stripe, Razorpay, PayPal - optional for Phase 1)**.
- ✓ Optimize app **performance (lazy loading, caching, database indexing)**.

Deliverables:

- Fully integrated reports, notifications, and payment module.
 - Optimized app performance.
-

Phase 6: Testing & Debugging (Weeks 18-20)

Goals:

- Conduct unit, integration, and usability testing.
- Fix major bugs and optimize app stability.

Tasks:

- ✓ Conduct **unit testing** (Flutter Test/Jest for React Native).
- ✓ Perform **integration testing** (API calls, data flow validation).
- ✓ Run **usability testing** with real users.
- ✓ Fix **UI/UX inconsistencies & security issues**.
- ✓ Test **performance across different devices & screen sizes**.

Deliverables:

- Bug-free, optimized application.
 - Test reports & bug fixes.
-

Phase 7: Deployment & Launch (Weeks 21-22)

Goals:

- Deploy app to the Play Store (Google).
- Set up analytics & marketing strategy.

Tasks:

- ✓ Register **Google Play Developer**.
- ✓ Generate **APK & IPA files** for submission.
- ✓ Implement **Firebase Analytics or Mixpanel** for usage tracking.
- ✓ Conduct **beta testing** using **TestFlight & Play Store Beta**.
- ✓ Fix any last-minute issues before full launch.
- ✓ Deploy app to **Google Play Store & Apple App Store**.

Deliverables:

- Successfully launched app on **Play Store & App Store**.
- Analytics tracking enabled.

Phase 8: Post-Launch Monitoring & Future Enhancements (Ongoing)

Goals:

- Gather user feedback & fix post-launch bugs.
- Plan future feature updates.

Tasks:

- ✓ **Monitor user feedback & crash reports** (Firebase Crashlytics).
- ✓ Release **patch updates & bug fixes**.
- ✓ Plan future features:
 - **AI-based stock prediction**.
 - **Multi-user/team collaboration**.
 - **Integration with digital payments**.

Deliverables:

- App updates & feature enhancements based on user feedback.
- Roadmap for future improvements.